08 · Hak Akses Data (RLS)

Row Level Security (RLS) membuat setiap pengguna hanya melihat baris data yang menjadi bagiannya, otomatis di semua chart, dashboard, SQL Lab, Studio, Asisten AI, dan pilihan filter. Ada dua cara; boleh dipakai bersamaan.

Cara	Cocok untuk	Diatur di
A. Aturan per role	semua perusahaan; jumlah kelompok sedikit (cabang, divisi)	Settings → Row Level Security
B. Otomatis dari profil pengguna	organisasi besar dengan LDAP + tabel profil pegawai (contoh: profil DJP)	`config\akses.yaml` + `{{ sps_akses(...) }}`

A. Aturan per role

1. Buat role per kelompok, mis. `Cabang Jakarta` (**Settings** → **List Roles** → **+**, salin izin `Gamma`).
2. Masukkan pengguna ke role itu (**Settings** → **List Users**).
3. **Settings** → **Row Level Security** → **+ Rule**:

Isian	Contoh
Rule name	`akses__latihan_penjualan__jakarta`
Filter type	**Regular**: berlaku hanya untuk role yang dipilih
Datasets	`latihan_penjualan` (bisa beberapa dataset sekaligus)
Roles	`Cabang Jakarta`
Group key	`cabang` (aturan dengan group key sama digabung dengan OR)
Clause	`nm_cabang = 'Cabang Jakarta'`

4. Uji dengan akun anggota role itu.

Jenis	Perilaku
Regular	syarat ditambahkan **hanya** untuk role terpilih; role lain melihat semua
Base	syarat ditambahkan untuk **semua kecuali** role terpilih. Pola aman: Base `1 = 0` (tidak melihat apa pun) + kecualikan role yang boleh melihat semua, lalu aturan Regular per kelompok

Ekstrak dan dataset hasil Persiapan data **mewarisi** aturan RLS sumbernya (13).

B. Otomatis dari profil pengguna (contoh: profil DJP)

Setiap pegawai hanya melihat data sesuai **unit kerja & jabatannya**, otomatis, berdasarkan profil yang dibaca dari database kepegawaian saat login LDAP. Aktif bila `AKSES_DATABASE` di `.env` diisi. Contoh di bawah adalah konfigurasi DJP (Oracle DMP: `PEGAWAI` + `DIM_KANTOR`); organisasi lain menyesuaikan query & atribut di `config\sps_akses.py` dan aturan di `config\akses.yaml`.

Cara kerja

```
LOGIN (LDAP)
  └─ sps_security.py → sps_akses.sinkron_profil(NIP)
       └─ Oracle: PEGAWAI + DIM_KANTOR  (query sama dengan engineN Mauth::cekuser)
       └─ simpan ke tabel sps_akses_profil (MySQL sps_superset)
       └─ cocokkan ke config/akses.yaml → lingkup + role "DJP ..."

SETIAP QUERY CHART / FILTER / SQL LAB
  └─ aturan RLS dataset:  {{ sps_akses(kd_kanwil='KD_KANWIL', kd_kpp='KD_KPP') }}
       └─ menjadi:  KD_KANWIL IN ('090') AND KD_KPP IN ('091')
```

Atribut profil

Atribut	Kolom Oracle	Contoh
`nip`	PEGAWAI.NIP9	060110956
`nip18`	PEGAWAI.NIP18	
`kd_kantor`	PEGAWAI.KD_KANTOR	
`kd_kpp`	PEGAWAI.KPPADM	091
`kd_kanwil`	DIM_KANTOR.KD_KANWIL	090
`tp_kantor`	DIM_KANTOR.TP_KANTOR	KPDJP / KANWIL / KPP / UPT
`kd_unor`	PEGAWAI.KD_UNIT_ORGANISASI	
`kd_jabatan`	PEGAWAI.KD_JAB_STRUKTURAL	
`kd_jabatan_fung`	PEGAWAI.KD_JAB_FUNGSIONAL	
`jabatan`	PEGAWAI.NAMA_JABATAN	Account Representative

Aturan bawaan (`config\akses.yaml`)

Aturan	Syarat	Lingkup data	Role
Kantor Pusat	tp_kantor = KPDJP	semua	DJP Pusat
Account Representative	jabatan = Account Representative	KPP-nya **dan** NIP-nya	DJP KPP
Pegawai Kanwil	tp_kantor = KANWIL	kanwilnya	DJP Kanwil
Pegawai KPP	tp_kantor = KPP	kanwil **dan** KPP-nya	DJP KPP
(tidak cocok)	—	**tidak ada data**	—

Aturan dibaca dari atas; yang **pertama cocok** dipakai. Satu aturan boleh menggabungkan **1 sampai banyak** atribut, baik di syarat (`kapan`) maupun di lingkup. Contoh aturan baru:

```
  - nama: Kepala Seksi Pengawasan di Kanwil Jakarta
    kapan: {tp_kantor: KANWIL, kd_kanwil: ["090", "091"], kd_jabatan: "1234"}
    lingkup: [kd_kanwil]
    peran: ["DJP Kanwil"]
```

Pengecualian per orang (menimpa aturan):

```
pengecualian:
  "123456789": {lingkup: {kd_kanwil: ["090", "091"]}, peran: ["DJP Kanwil"]}
  "987654321": {lingkup: semua, peran: ["DJP Pusat"]}
```

Perubahan `akses.yaml` berlaku pada **login berikutnya** pegawai tsb, tanpa restart.

Syarat pada data ("filter wajib")

Agar data bisa dibatasi, **dataset wajib punya kolom** untuk setiap atribut di lingkup pemakainya. Di dashboard as code, kolom itu didaftarkan di `akses`:

```
Dataset(
    key="ppm_per_kpp",
    sql="SELECT KD_KANWIL, KD_KPP, NIP_AR, ... FROM ...",
    columns=[...],
    akses={"kd_kanwil": "KD_KANWIL", "kd_kpp": "KD_KPP", "nip": "NIP_AR"},
)
```

Nilai `akses`	Arti
`{"kd_kanwil": "KD_KANWIL", ...}`	Dibatasi sesuai profil (data regional)
`{}` (bawaan)	Data **nasional**: hanya lingkup `semua` (Pusat & Admin)
`None`	Data referensi, tidak dibatasi (mis. pilihan tahun/bulan)

Builder otomatis membuat aturan RLS bernama `akses__<nama dataset>` (lihat **Settings → Row Level Security**). Dataset yang dibuat manual lewat UI: tambahkan aturan RLS sendiri dengan klausa yang sama.

Prinsip **gagal = tertutup**: kolom yang diwajibkan tidak ada, nilai profil kosong, profil tidak ditemukan, atau tidak ada aturan cocok → `1 = 0` (tidak ada data). Admin Superset selalu `1 = 1`.

Hak akses dashboard

Fitur `DASHBOARD_RBAC` aktif: dashboard bisa dibatasi per role lewat dashboard → **Edit properties** → **Roles**. Kosong = mengikuti akses dataset.

Dashboard (profil DJP)	Role
Beranda DJP	DJP Pegawai (semua pegawai), DJP Pusat
Home Nasional	DJP Pusat

Hal penting

- **Role bawaan pengguna sebaiknya** `Gamma` (`LDAP_DEFAULT_ROLE` di `.env`). Role `Alpha` bisa membuat dataset baru langsung dari tabel sumber sehingga **melewati** hak akses. Berikan Alpha hanya ke analis tepercaya.

- SQL Lab hanya untuk Admin (atau role `sql_lab`). `RLS_IN_SQLLAB` aktif, tetapi query bebas tetap bisa membaca tabel yang tidak punya dataset ber-RLS.
- Melihat profil yang tersimpan (cara B): tabel `sps_akses_profil` di metadb (`docker compose exec metadb psql -U superset -d sps_superset -c "select * from sps_akses_profil limit 5"`).
- Log setiap login mencatat aturan yang dipakai: `ALDx akses 060110956: aturan 'Kantor Pusat', lingkup semua, role [...]`.